CORE103 Computational Thinking
PW10 - Recursion I



Department of Computer Science

Dhanani School of Science and Engineering

Habib University

Fall 2026

Copyright @ 2026 Habib University

Contents

1 Introduction **2**
- 1.1 Instructions . . . 2
- 1.2 Marking scheme . . . 2
  - 1.2.1 Objectives . . . 2
- 1.3 Late submission policy . . . 2
- 1.4 Use of AI . . . 2
- 1.5 Viva . . . 3

2 Prerequisites **4**
- 2.1 Assert Statements . . . 4
- 2.2 Pytest . . . 4

3 Exercises **6**
- 3.1 Collatz Conjecture . . . 6
- 3.2 Pattern . . . 8
- 3.3 Counting with a Buzz Table . . . 10
- 3.4 Fibonacci Numbers . . . 11
- 3.5 Number Pattern . . . 13
- 3.6 Caffeine . . . 14
- 3.7 Paper Cup Pattern . . . 17
- 3.8 Lift Stops . . . 19
- 3.9 Multiplication Table . . . 20
- 3.10 Slow Reveal . . . 22

Introduction

1.1 Instructions

- This PW must be submitted online via CANVAS. You are required to submit a zip file that contains all the *.py* files.
- The zip file should be named as *PW10_aa123456.zip* where *aa123456* will be replaced with your student id.
- **Files that don't follow the appropriate naming convention will not be graded.**

1.2 Marking scheme

This PW will be marked out of 10.

1.2.1 Objectives

Following are the objectives of this practice worksheet:

- Understand the fundamental concept of recursion in programming.
- Differentiate between recursive functions and iterative solutions.
- Explain the structure of recursive algorithms, including base cases and recursive calls.
- Write simple recursive functions to solve basic problems.

1.3 Late submission policy

There is no late submission policy for the PW.

1.4 Use of AI

Taking help from any AI-based tools such as ChatGPT is strictly prohibited and will be considered plagiarism.

1.5 Viva

Course staff may call any student for Viva to provide an explanation for their submission.

Prerequisites

2.1 Assert Statements

Assert statements are crucial tools in programming used to verify assumptions and test conditions within code. For instance, in Python, an assert statement is typically as follows:

```
    assert condition, message
```

For example, in a function calculating the square root, in order to ensure that the input value is non-negative, one might add the following assert statement:

```
assert x >= 0, "x should be a non-negative number"
```

If the condition evaluates to False, an AssertionError is raised, highlighting the issue and its corresponding message.

2.2 Pytest

Pytest is a popular testing framework in Python known for its simplicity and power in facilitating efficient and scalable testing for applications and software. In order to use Pytest, one first needs to install it and create test functions prefixed with *test_* in a Python file. To install Pytest, you can write the following command on the terminal:

```
    pip install pytest
```

After installing Pytest, one can execute all the tests by typing the following command on the terminal:

```
    pytest
```

If you want to test any particular file, then you will have to specify the filename as follows:

```
    pytest <filename>.py
```

If all the tests pass successfully, then you should receive a similar output.

```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
========================================================================================== test session starts ========
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py ..........

========================================================================================== 10 passed in 0.06s =========
```

However, if all the test fail, then you should receive a similar output.

```
PS C:\Work\Spring2024\DSA\Lab1> pytest test_Question1.py
============================================================================ test session starts ============================================================================
platform win32 -- Python 3.11.2, pytest-7.4.4, pluggy-1.3.0
rootdir: C:\Work\Spring2024\DSA\Lab1
collected 10 items

test_Question1.py FFFFFFFFFF

================================================================================= FAILURES ==================================================================================
_____________________________________ test_question1[lst10-lst20-e54dc15ccafa608142ffa6340b035c327f972e24882052cfc3345f240f4ee237] ____
```

Exercises

3.1 Collatz Conjecture

This question should be implemented in the file ***q1.py****.*

Problem Description

The Collatz conjecture is a famous unsolved mathematical problem. It states that starting from any positive integer and repeatedly applying the following rules will eventually produce **1**:

- If the number is **even**, divide it by **2**.
- If the number is **odd**, multiply it by **3** and add **1**.

For example, starting with **5**, the sequence is:

5 -> 16 -> 8 -> 4 -> 2 -> 1

Your task is to generate the Collatz sequence using **recursion**.

Function Description

Write a recursive function **verify_collatz** that takes one integer parameter, starting_number, and prints the Collatz sequence starting from that number.

Print each number on a separate line, including the starting number and the final **1**. Stop when the number reaches **1**. The function prints the sequence and returns nothing.

Sample Interaction

```
>>> verify_collatz(1)
1

>>> verify_collatz(5)
5
16
8
4
2
1

>>> verify_collatz(52)
52
26
13
40
20
10
5
16
8
4
2
1
```

Sample Interaction Explanation

In the first example, the starting number is already **1**, so only **1** is printed.

In the second example, **5** is odd, so it becomes **16**. The number is then repeatedly halved until it reaches **1**.

In the third example, the sequence starts at **52** and follows the even and odd rules until it reaches **1**.

Constraints

- $1 \leq$ starting_number ≤ 1000.
- You must use **direct recursion** within **verify_collatz**.
- You are not allowed to define or use helper functions.
- Loops, comprehensions, and other forms of iteration are not allowed.
- You may not use global variables.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q1.py
```

3.2 Pattern

This question should be implemented in the file ***q2.py****.*

Problem Description

The Computing Society is creating a right-aligned triangle of asterisks for a terminal display. Each row contains one more asterisk than the previous row. Your task is to print this pattern using **recursion**.

Function Description

Write a function **pattern** that takes one parameter, number_of_rows, representing the number of rows in the triangle. The first row contains one asterisk, the second contains two, and so on.

The triangle must be right-aligned, with exactly one space between consecutive asterisks and no trailing spaces. Follow the alignment shown in the sample interactions.

You must define a **recursive helper function** and call it from **pattern**. You may choose the helper's name and parameters. The function prints the pattern and returns nothing.

Special Cases

- If number_of_rows is **0**, print nothing.
- If number_of_rows is greater than **50**, print exactly **Program can't handle this input**.
- If number_of_rows is negative or is not an integer, print exactly **Invalid input**.
- Boolean values are considered invalid inputs.

Sample Interaction

```
>>> pattern(4)
      *
    * *
  * * *
* * * *

>>> pattern(2)
  *
* *

>>> pattern(1)
*

>>> pattern(0)

>>> pattern(-2)
Invalid input

>>> pattern("four")
Invalid input

>>> pattern(51)
Program can't handle this input
```

Sample Interaction Explanation

In the first example, the pattern has four rows, containing one, two, three, and four asterisks respectively. The leading spaces decrease by two in each successive row. For **0**, nothing is printed. Invalid inputs and inputs exceeding the supported limit produce the corresponding error messages.

Constraints

- The supported range is $0 \leq$ number_of_rows ≤ 50.
- You must define and call a **recursive helper function**.
- Loops, comprehensions, and other forms of iteration are not allowed.
- You may not use global variables.
- String concatenation and repetition are allowed.
- Print each row on a separate line, with exactly one space between asterisks and no trailing spaces.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q2.py
```

3.3 Counting with a Buzz Table

*This question should be implemented in the file **q3.py**.*

Problem Description

During Societies Week, the Computing Society organizes a counting game. Students count from **1** to a given number, replacing every multiple of **5** with the word **Buzz**. Your task is to print this sequence using **recursion**.

Function Description

Write a function **count_buzz** that takes one parameter, last_number, and prints the sequence from **1** to last_number, replacing every multiple of **5** with **Buzz**. Print the sequence on a single line, separating items with a comma and a space, without a trailing comma or spaces. The function returns nothing. You may define a recursive helper function if needed.

Sample Interaction

```
>>> count_buzz(13)
1, 2, 3, 4, Buzz, 6, 7, 8, 9, Buzz, 11, 12, 13

>>> count_buzz(5)
1, 2, 3, 4, Buzz

>>> count_buzz(1)
1

>>> count_buzz("thirteen")
Error: last_number must be a positive integer.

>>> count_buzz(0)
Error: last_number must be a positive integer.

>>> count_buzz(-3)
Error: last_number must be a positive integer.
```

Sample Interaction Explanation

In the first example, **5** and **10** are replaced with **Buzz**. The other examples demonstrate shorter sequences and invalid inputs.

Constraints

- Valid inputs are integers satisfying $1 \leq$ last_number ≤ 100.
- For invalid inputs, print exactly **Error: last_number must be a positive integer.**
- Boolean values are considered invalid inputs.
- You must use **recursion**. A recursive helper function is allowed.
- Loops, comprehensions, and other forms of iteration are not allowed.
- Global variables are not allowed.
- Print the sequence on one line, with exactly a comma and a space between items, and no trailing comma or spaces.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q3.py
```

3.4 Fibonacci Numbers

*This question should be implemented in the file **q4.py**.*

Problem Description

During a CORE103L revision session, students explore the Fibonacci sequence. The sequence begins with **1** and **1**, and each subsequent number is the sum of the previous two numbers. For example, the first seven numbers are **1, 1, 2, 3, 5, 8, 13**. Your task is to print this sequence using **recursion**.

Function Description

Write a recursive function **print_fibb** that takes an integer parameter, number_of_terms, and prints the first number_of_terms Fibonacci numbers, one number per line. The function prints the sequence in order and returns nothing.

You may use additional parameters with default values, as practised in the seminar, or define a recursive helper function if needed.

Sample Interaction

```
>>> print_fibb(3)
1
1
2

>>> print_fibb(5)
1
1
2
3
5

>>> print_fibb(1)
1

>>> print_fibb(7)
1
1
2
3
5
8
13
```

Sample Interaction Explanation

In the first example, the function prints the first three Fibonacci numbers: **1**, **1**, and **2**. In the second example, it prints five numbers, ending with **5**. When number_of_terms is **1**, only the first number is printed.

Constraints

- $1 \leq$ number_of_terms ≤ 100.
- You must use **recursion**.
- Additional parameters with default values are allowed.
- A recursive helper function is allowed.
- Loops, comprehensions, and other forms of iteration are not allowed.
- Global variables are not allowed.
- Print each Fibonacci number on a separate line.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q4.py
```

3.5 Number Pattern

This question should be implemented in the file ***q5.py****.*

Problem Description

During a Computing Society activity, students are creating number patterns on a terminal screen. One pattern forms a triangle where each row starts at **1** and contains one more number than the previous row. Your task is to print this triangle using **recursion**.

Function Description

Write a function **print_number** that takes one integer parameter, number_of_rows, representing the number of rows in the triangle. The first row contains **1**, the second contains **1 2**, and so on, until the last row contains the numbers from **1** to number_of_rows.

Print each row on a separate line, with exactly one space between numbers and no leading or trailing spaces. The function prints the pattern and returns nothing. You must define a **recursive helper function** and call it from **print_number**. You may choose the helper function's name and parameters.

Sample Interaction

```
>>> print_number(4)
1
1 2
1 2 3
1 2 3 4

>>> print_number(1)
1

>>> print_number(5)
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Sample Interaction Explanation

In the first example, the function prints four rows, with each row containing one more number than the previous row. In the second example, only one row is printed. The third example prints five rows, ending with **1 2 3 4 5**.

Constraints

- $1 \leq$ number_of_rows ≤ 100.
- You must use **recursion**.
- You must define a recursive helper function and call it from **print_number**.
- Loops, comprehensions, and other forms of iteration are not allowed.
- Global variables are not allowed.
- Print each row on a separate line, with exactly one space between numbers and no leading or trailing spaces.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q5.py
```

3.6 Caffeine

*This question should be implemented in the file **q6.py**.*

Problem Description

You have opened a coffee stall on campus and want to encourage students to try your favorite drink, Iced Matcha Latte. It is available in all sizes with a discount of Rs. 50, while other drinks are available only in specific sizes. Your task is to process customer orders using **recursion**.

Function Description

Write a recursive function **caffeine** that takes one integer parameter, number_of_orders, representing the total number of orders. For each order:

1. Ask the customer to select a size (**L**, **M**, or **S**) using:

   ```
   Enter drink size (L/M/S):
   ```

2. Display the available drink options for that size, as listed below.

3. Ask the customer to select a drink using:

   ```
   Flavor:
   ```

4. Calculate the price based on the selected size. Subtract Rs. 50 if the drink is **Iced Matcha Latte**.

5. Print the order details in the following format, followed by one empty line:

```
Order <number>: <Size> <Flavor> --- <Price>
```

Process exactly number_of_orders orders, numbering them from **1** onward. The function prints the order details and returns nothing. You must use **direct recursion** within **caffeine**; helper functions are not allowed.

Available Drinks

For every size, first display:

```
Choose from one of the following flavors:
--Iced Matcha Latte
```

Then display the additional options for the selected size:

- **Large (L):**

```
--Boba Tea
--French Vanilla
--Peppermint
--Masala Chai
```

- **Medium (M):**

```
--Cappuccino
--Flat White
--Espresso
--Black Coffee
--Pumpkin Spice Latte
```

- **Small (S):**

```
--Caramel Latte
--Vanilla Latte
--Americano
--Frappe
--Ristretto
--Turkish Coffee
```

Drink Prices

Size	Base Price (Rs.)
Large (L)	225
Medium (M)	195
Small (S)	160

Note: Iced Matcha Latte receives a discount of Rs. 50 in every size.

Sample Interaction

In the sample, `[ Input -> x ]` represents user input, not program output.

```
>>> caffeine(2)
Enter drink size (L/M/S):
[ Input -> L ]
Choose from one of the following flavors:
--Iced Matcha Latte
--Boba Tea
--French Vanilla
--Peppermint
--Masala Chai
Flavor:
[ Input -> Masala Chai ]
Order 1: Large Masala Chai --- 225

Enter drink size (L/M/S):
[ Input -> M ]
Choose from one of the following flavors:
--Iced Matcha Latte
--Cappuccino
--Flat White
--Espresso
--Black Coffee
--Pumpkin Spice Latte
Flavor:
[ Input -> Iced Matcha Latte ]
Order 2: Medium Iced Matcha Latte --- 145
```

Sample Interaction Explanation

In the first order, the customer selects a Large Masala Chai for Rs. 225. In the second order, the customer selects a Medium Iced Matcha Latte, receiving a Rs. 50 discount on the base price of Rs. 195.

Constraints

- $1 \leq$ number_of_orders ≤ 20.
- You must use **direct recursion** within **caffeine**.
- Helper functions are not allowed.
- Loops, comprehensions, and other forms of iteration are not allowed.
- Global variables are not allowed.
- The size will always be **L**, **M**, or **S**.
- The customer will always select a valid drink using the exact spelling shown in the menu. Input validation is not required.

- Print orders in increasing order, starting from **1**.
- Print exactly one empty line after each completed order.
- The prompts, menu entries, and order details must follow the specified formats.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q6.py
```

3.7 Paper Cup Pattern

This question should be implemented in the file ***q7.py****.*

Problem Description

During orientation week, a campus refreshment stall wants to display a stylized paper cup on its welcome screen. The pattern is widest at the top and gradually narrows to a single asterisk at the bottom. Your task is to print this pattern using **recursion**.

Function Description

Write a function **cup** that takes one integer parameter, n, representing the number of asterisks in the top row. The function prints exactly n rows, with each subsequent row containing one fewer asterisk until only one remains.

The pattern must be aligned as shown in the sample interactions, with exactly one space between consecutive asterisks. Trailing spaces at the end of each row are allowed. The function prints the pattern and returns nothing.

You must define a **recursive helper function** and call it from **cup**. You may choose the helper function's name and parameters.

Constraints

- $1 \leq n \leq 50$.
- You must define and call a **recursive helper function**.
- Loops (**for** or **while**), comprehensions, and other forms of iteration are not allowed.
- String concatenation and repetition are allowed.
- Global variables are not allowed.
- Each row must be printed on a separate line.
- Exactly one space must separate consecutive asterisks. Trailing spaces are allowed.

Sample Interaction

```
>>> cup(4)
* * * *
 * * *
  * *
   *

>>> cup(2)
* *
 *

>>> cup(1)
*
```

Sample Interaction Explanation

In the first example, **cup(4)** prints four rows. Each row contains one fewer asterisk than the previous row, and the pattern gradually narrows toward the bottom.

Row	Leading Spaces	Asterisks	Printed Line
1 (top)	0	4	"* * * * "
2	1	3	" * * * "
3	2	2	" * * "
4 (bottom)	3	1	" * "

In the second example, **cup(2)** prints two rows. The first row contains two asterisks, while the second contains one asterisk.

In the third example, **cup(1)** prints a single asterisk.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q7.py
```

3.8 Lift Stops

This question should be implemented in the file ***q8.py****.*

Problem Description

During a campus programming activity, students are building a display for a limited-stop lift that stops only at even-numbered floors. The ground floor is **0**, while basement floors are represented by negative numbers. Your task is to display the lift stops within a given range using **recursion**.



Function Description

Write a recursive function **print_lift_stops** that takes two integer parameters, first_floor and last_floor, and prints all even-numbered floors within this range (inclusive) in increasing order.

Print the floors on a single line, separated by a comma and a space, without a trailing comma or spaces. If there are no even-numbered floors in the range, print nothing. The function prints the sequence and returns nothing.

Sample Interaction

```
>>> print_lift_stops(0, 6)
0, 2, 4, 6

>>> print_lift_stops(1, 5)
2, 4

>>> print_lift_stops(-3, 3)
-2, 0, 2

>>> print_lift_stops(2, 3)
2
```

```
>>> print_lift_stops(3, 4)
4

>>> print_lift_stops(1, 1)

>>> print_lift_stops(0, 0)
0
```

Sample Interaction Explanation

In the first example, the lift stops at **0**, **2**, **4**, and **6**. In the third example, the range includes basement floors, so negative even numbers are also printed. For **print_lift_stops(1, 1)**, nothing is printed because the range contains no even-numbered floors.

Constraints

- $-10 \leq$ first_floor $\leq$ last_floor ≤ 100.
- Both parameters are integers.
- You must use **direct recursion** within **print_lift_stops**.
- Helper functions are not allowed.
- Loops, comprehensions, and other forms of iteration are not allowed.
- Global variables are not allowed.
- Print the floors on a single line, separated by a comma and a space, without a trailing comma or spaces.
- If the range contains no even-numbered floors, print nothing.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q8.py
```

3.9 Multiplication Table

*This question should be implemented in the file **q9.py**.*

Problem Description

Your younger cousin is struggling to memorize multiplication tables. Since you are learning recursion in CORE103L, you decide to write a program to help your cousin practise. Your task is to print a multiplication table using **recursion**.

Function Description

Write a recursive function **multiplication_table** that takes two integer parameters, number and last_multiplier. The function prints the multiplication table of number, starting from multiplication by **1** and ending at last_multiplier, inclusive.

Print each calculation on a separate line in the format **number * multiplier = product**, with exactly one space around each operator and no trailing spaces. The function prints the table and returns nothing.

Sample Interaction

```
>>> multiplication_table(5, 10)
5 * 1 = 5
5 * 2 = 10
5 * 3 = 15
5 * 4 = 20
5 * 5 = 25
5 * 6 = 30
5 * 7 = 35
5 * 8 = 40
5 * 9 = 45
5 * 10 = 50

>>> multiplication_table(3, 4)
3 * 1 = 3
3 * 2 = 6
3 * 3 = 9
3 * 4 = 12

>>> multiplication_table(7, 1)
7 * 1 = 7
```

Sample Interaction Explanation

In the first example, the function prints the multiplication table of **5**, from **5 * 1 = 5** to **5 * 10 = 50**. In the second example, it prints the first four multiples of **3**. In the third example, only one calculation is printed because last_multiplier is **1**.

Constraints

- $1 \leq$ number ≤ 100.
- $1 \leq$ last_multiplier ≤ 100.
- You must use **direct recursion** within **multiplication_table**.
- Helper functions are not allowed.
- Loops, comprehensions, and other forms of iteration are not allowed.

- Global variables are not allowed.
- Print each calculation on a separate line, with exactly one space around each operator and no trailing spaces.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q9.py
```

3.10 Slow Reveal

This question should be implemented in the file ***q10.py****.*

Problem Description

During Societies Week, a student society wants to display a surprise announcement on a terminal screen. Instead of showing the entire message at once, the screen reveals one additional character on each line until the complete message appears. Your task is to create this effect using **recursion**.

Function Description

Write a recursive function **slow_reveal** that takes one string parameter, message. The function first prints the first character, then the first two characters, and continues until the complete message is printed.

Print each partial message on a separate line, preserving all characters, including spaces and punctuation. The function prints the output and returns nothing. If message is empty, print nothing.

Sample Interaction

```
>>> slow_reveal("Pizza!")
P
Pi
Piz
Pizz
Pizza
Pizza!

>>> slow_reveal("Hi")
H
Hi

>>> slow_reveal("A")
A
```

```
>>> slow_reveal("")
```

Sample Interaction Explanation

In the first example, **slow_reveal("Pizza!")** prints one additional character on each line until the complete message is revealed. In the second and third examples, the function prints two lines and one line, respectively. In the final example, nothing is printed because the string is empty.

Constraints

- $0 \leq$ **len(message)** ≤ 100.
- The string may contain letters, digits, spaces, and punctuation.
- You must use **direct recursion** within **slow_reveal**.
- Helper functions are not allowed.
- Loops, comprehensions, and other forms of iteration are not allowed.
- Global variables are not allowed.
- String indexing, slicing, concatenation, and repetition are allowed.
- Preserve all characters and their original order.
- Print each partial message on a separate line. If the string is empty, print nothing.

Testing

In order to test your function, type the following command on the terminal:

```
pytest tests/test_q10.py
```